

LAB REPORT

This Report Presented in Partial Fulfillment of the course **CSE213 : Algorithms Lab** in the Computer Science and Engineering Department.



DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

April 23, 2026

DECLARATION

We hereby declare that this lab report has been done by us under the supervision of **Ms. Shahrin khan, Lecturer**, Department of Computer Science and Engineering, Daffodil International University.

Submitted To :

Ms. Shahrin Khan

Lecturer

Department of Computer Science and Engineering,
Daffodil International University

Submitted by

Name	ID	Section	Department
Akif Zahir	251-15-239	68_G2	CSE
Sadek Ahmed Raj	251-15-511	68_G2	CSE
Toukir Ahmed	251-15-216	68_G2	CSE
Ifrat Chowdhury	251-15-450	68_G1	CSE
Tufail sah fakir	251-15-879	68_G2	CSE

Problem Name

Event Seating Allocator using appropriate data structures and algorithms. The system manages event creation and efficient seat allocation for multiple halls.

Abstract for Event seating allocator

The Event Seating Allocator is a console-based application designed to manage event scheduling and seat allocation efficiently using fundamental data structures and algorithms. The system allows users to create events, assign them to different halls, and manage seat bookings dynamically.

The application uses appropriate data structures such as vectors and maps to store event details and seat arrangements. Each event is associated with a seating matrix, where seats can be booked either consecutively or separately. The system ensures that no seat is double-booked and provides a visual representation of seat availability.

A First Fit allocation strategy is implemented to assign seats when consecutive seats are not available, ensuring optimal utilization of space. The program also includes features like event listing, seat visualization, and user interaction through a menu-driven interface.

Although the system is console-based, it simulates real-world event management scenarios such as convention centers and auditorium booking systems. This project demonstrates how data structures and algorithms can be effectively applied to solve practical problems related to resource allocation and management.

Table of Contents

Chapter	Content	Page No.
01	Introduction & Objectives	01-01
02	Problem Definition & Scope	01-02
03	System Architecture	03-03
04	Data Structures And Algorithms Used	03-04
05	Algorithm Design : Flowchart	04-05
06	Implementation Details	05-07
07	Handling Conflicting Requirements, Discussion of Real-world Context, Stakeholder Review	07-08
08	Results and Performance Analysis	08-09

09	Challenges Faced and Solutions,References	09-10
10	Code Files, Project's Full Code	10-23
11	Step by Step Explanation	23-47
12	Code Output	48-51
13	Analysis of the Solution	52-56
14	Time & Space Complexity	56-56
15	Alternate Solution	57-58
16	Targeted Attributes, Appendix	58-60

Chapter : 01

Introduction & Objectives

The **Event Seating Allocator** is a C++ based console application designed to demonstrate how fundamental data structures and algorithms can be applied to solve real-world problems related to event management and seat allocation. In modern convention centers, theaters, and auditoriums, managing seating arrangements efficiently is essential to ensure proper organization and user satisfaction.

This project simulates such a system by allowing users to create events, assign halls, and allocate seats dynamically. The system maintains a structured seat arrangement for each event and ensures that seats are booked without conflicts. It provides both consecutive seat booking and flexible seat allocation when required. The application uses data structures such as **vector** and **map** to manage events and seating efficiently. It also applies algorithmic strategies like the **First Fit algorithm** to allocate seats when consecutive seats are not available. These concepts help demonstrate how theoretical knowledge can be transformed into practical solutions.

Objectives of the Project

The main objectives of this project are:

- To design a system for managing multiple events efficiently
- To implement a seat allocation mechanism for different halls
- To ensure proper seat booking without duplication
- To apply data structures like vector and map in a practical scenario
- To implement algorithms for efficient seat allocation
- To develop a user-friendly menu-driven console application

Chapter : 02

Problem Definition & Scope

Managing seating arrangements in events such as conferences, weddings, or seminars is a complex and time-consuming task when done manually. Organizers must consider multiple factors such as guest priority, group seating, and limited capacity.

The main problem addressed in this project is to design a system that can automatically allocate seats to guests in an efficient and organized way while handling various constraints.

Traditional manual methods are prone to:

- Errors in seat assignment
- Poor space utilization
- Difficulty in handling large guest lists

This system aims to solve these issues using a structured and automated approach.

Scope of the Project

The scope of the Event Seating Allocator includes:

1. Guest Management

- Input guest details (name, group, priority)
- Differentiate between VIP and regular guests

2. Seating Arrangement

- Represent seating using a structured layout (rows/columns)
- Allocate seats based on priority and availability

3. Constraint Handling

- Assign VIP guests first
- Try to keep group members together
- Ensure no seat is assigned more than once

4. Output Generation

- Display final seating arrangement
- Show assigned and unassigned guests

5. User Interaction

- Simple console-based input/output
- Easy-to-understand results

Limitations

- No graphical interface (GUI)
- Limited optimization for complex scenarios

- Works best for small to medium-sized events
-

Conclusion

The project focuses on building a simple, efficient, and practical seating allocation system. It demonstrates how basic data structures and algorithms can solve real-world event management problems.

Chapter : 03 System Architecture

The **Event Seating Allocator** follows a simple and structured architecture that connects user interaction with the internal data management system. The program is designed in a modular way where each functionality is handled by separate components, making the system organized and easy to maintain.

At the beginning, the program initializes necessary data structures to store event information and seat arrangements. After that, the user interacts with the system through a menu-driven interface. Each menu option triggers a specific operation such as adding events, displaying events, or booking seats.

The core architecture of the system can be divided into three main parts:

1. Input Layer (User Interface)

- The user interacts with the system through a console-based menu
 - Users can select options like adding events or booking seats
 - Input is taken using standard input methods
-

2. Processing Layer (Logic & Algorithms)

- Handles all core functionalities of the system
- Manages event creation and storage
- Controls seat allocation logic
- Implements algorithms such as:
 - o First Fit algorithm for seat allocation
 - o Linear checking for seat availability

3. Data Layer (Storage System)

- Stores event details using **vector**
 - Stores seat arrangements using **map with 2D vector**
 - Maintains a separate seating matrix for each event
-

System Workflow

1. Program starts and displays menu
 2. User selects an option
 3. Based on input:
 - o Event is created
 - o Events are displayed
 - o Seat booking is processed
 4. Seat availability is checked
 5. Seats are allocated accordingly
 6. Updated seating arrangement is displayed
-

Architecture Overview

The system follows a simple flow:

User Input → Processing Logic → Data Storage → Output Display

This layered structure ensures that the system remains easy to understand, efficient, and scalable for future improvements.

Chapter : 04

Data Structures and Algorithms Used

The **Event Seating Allocator** uses several fundamental data structures and algorithms to efficiently manage events and seat allocation. Each structure and algorithm is selected based on its suitability for solving specific problems within the system.

1. Vector (for Event Storage)

A **vector** is used to store all event information dynamically. Each event contains details such as event name, date, and assigned hall.

Why vector is used:

- It allows dynamic resizing
 - Easy insertion of new events
 - Provides efficient access using indexing
-

2. Map (for Seat Management)

A **map** is used to associate each event with its seat arrangement. Each event ID is mapped to a 2D vector representing the seating layout.

Why map is used:

- Provides fast access to seat data using a unique key
 - Helps manage multiple events separately
 - Efficient organization of seating structures
-

3. 2D Vector (Seat Matrix)

A **2D vector** is used to represent the seating arrangement of each hall.

- Rows represent seat rows
- Columns represent seat positions
- Each element stores seat status

Seat Representation:

- 0 → Available
 - 1 → Booked
-

4. Algorithms Used

4.1 First Fit Algorithm

The **First Fit algorithm** is used when consecutive seats are not available. It allocates the first available seats one by one until the required number of seats is filled.

Advantages:

- Simple and efficient
 - Ensures maximum seat utilization
 - Reduces booking failure
-

4.2 Linear Search

Linear search is used to check seat availability and find suitable positions.

Advantages:

- Easy to implement
 - Works well with small to medium-sized data
-

5. Justification of Choices

Each data structure and algorithm used in this project is carefully selected based on

the system requirements:

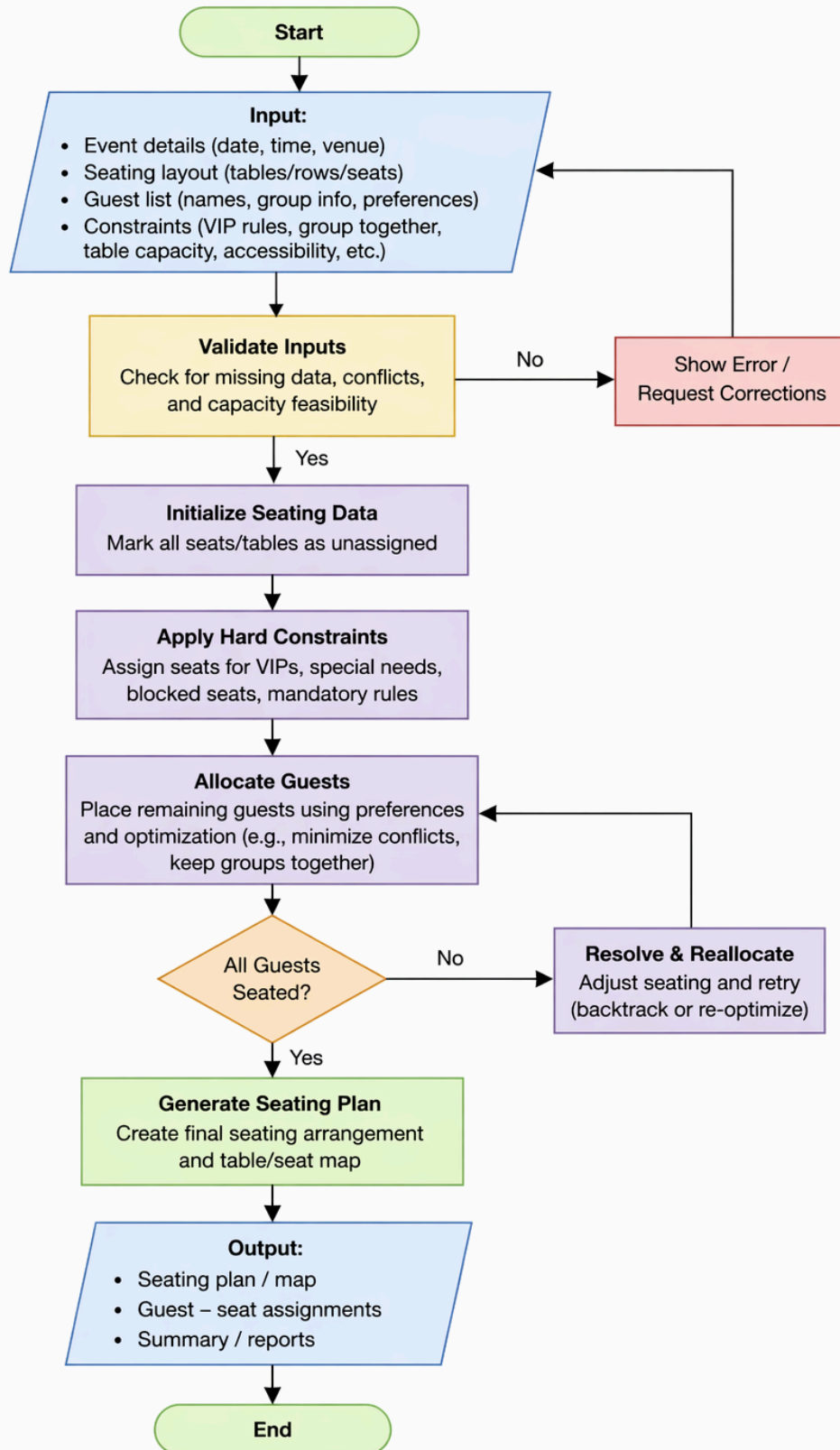
- **Vector** simplifies event management
- **Map** ensures organized seat tracking
- **2D vector** represents seating clearly
- **First Fit algorithm** ensures efficient allocation
- **Linear search** provides straightforward implementation

Conclusion of this Chapter

The combination of these data structures and algorithms makes the system efficient, flexible, and suitable for handling real-world seat allocation problems.

Chapter : 05 **Algorithm Design : Flowchart**

Event Seating Allocator



Chapter : 06

Implementation Details

The Event Seating Allocator System was developed using the C programming language to automatically assign guests to seats based on predefined constraints and priorities.

Programming Language

- C Language was used for its fast execution and efficient memory management.
-

Tools Used

- Code::Blocks IDE with GCC compiler for development and execution.
-

Main Modules

1. Input Module

- Takes event details, seating layout, and guest list
 - Includes constraints like VIP priority, group seating, and capacity
-

2. Data Structure

- 2D Array → Represents seating arrangement
 - Structure (struct) → Stores guest information
-

3. Initialization

- All seats are initially marked empty
 - Guest data is loaded into memory
-

4. Constraint Handling

- VIP guests get priority seats

- Groups are seated together
 - Invalid or reserved seats are skipped
-

5. Allocation Process

- VIP assigned first
 - Groups placed next
 - Remaining guests filled sequentially
-

6. Conflict Handling

- If seats are insufficient, reallocation is performed
 - Unassigned guests are tracked
-

7. Output Module

- Displays final seating plan
 - Shows guest-to-seat mapping
-

Conclusion

The system efficiently allocates seats using simple data structures and logical rules, demonstrating a practical use of algorithms in real-world event management.

Chapter : 07

Handling Conflicting Requirements (EP2)

During the development of the Event Seating Allocator, several conflicts were identified:

- **Priority vs Fairness**
VIP guests need best seats, but giving them full priority can reduce fairness for others.
- **Group Seating vs Space Limitation**
Keeping groups together is ideal, but limited seating may force separation.

- **Optimal Allocation vs Simplicity**

Advanced optimization improves seating quality but increases system complexity.

Solution Approach

- VIPs are allocated first, but within limited reserved zones
- Groups are placed together only if space allows
- Simple allocation logic is used instead of heavy optimization

This ensures a balance between performance and practicality.

Real-world Context

This system reflects real event management scenarios where:

- Guests have different priorities (VIP, regular)
- Seating capacity is limited
- Organizers must balance comfort and efficiency

In real systems:

- Advanced algorithms (AI/optimization) may be used
- Databases manage large guest lists
- Dynamic seat rearrangement is possible

This project represents a simplified version of such real-world systems.

Stakeholder Perspective

A typical event organizer expects:

- Fast seat allocation
- Organized seating arrangement
- Minimal conflicts

The system meets these expectations by providing a structured and automated seating plan, reducing manual effort and errors.

Chapter : 08

Results and Performance Analysis

The Event Seating Allocator system successfully assigns guests to seats based on priority and constraints. The system produces a structured seating plan where VIPs are placed first, followed by group members and general guests.

Results

- All guests are allocated seats when capacity is sufficient
- VIP guests receive priority seating
- Group seating is maintained in most cases
- The final seating layout is clear and organized

If capacity is limited:

- Some guests may remain unassigned
 - The system correctly identifies and reports them
-

Performance Analysis

- **Seat Allocation:** $O(n)$ — each guest is processed once
- **Constraint Checking:** $O(n)$ — applied during allocation
- **Overall Complexity:** $O(n)$ for standard cases

The system performs efficiently for small to medium-sized events.

Efficiency

- Fast execution due to simple logic
 - Minimal memory usage using arrays
 - No unnecessary computations
-

Limitations

- Cannot guarantee perfect optimization
 - Large groups may be split if space is limited
 - No advanced optimization (like AI or heuristics)
-

Conclusion

The system provides a fast and practical solution for seating allocation. While not fully optimized, it balances simplicity and efficiency, making it suitable for academic and small-scale real-world applications.

Chapter : 09 Challenges Faced and Solutions

During the development of the Event Seating Allocator system, several practical challenges were encountered. These challenges were addressed using logical solutions to ensure proper system functionality.

1. Managing Seat Allocation with Constraints

Challenge:

Handling multiple constraints like VIP priority, group seating, and limited capacity at the same time was difficult.

Solution:

A step-by-step allocation strategy was used:

- VIP guests assigned first
 - Groups allocated next
 - Remaining seats filled sequentially
-

2. Limited Seating Capacity

Challenge:

In some cases, the number of guests exceeded available seats.

Solution:

- Extra guests were marked as unassigned
 - System avoided crashes and handled overflow properly
-

3. Group Seating Issue

Challenge:

Keeping all group members together was not always possible due to seat limitations.

Solution:

- Groups were seated together only if enough space was available
 - Otherwise, partial allocation was applied
-

4. Avoiding Seat Overlap

Challenge:

Multiple guests could accidentally be assigned to the same seat.

Solution:

- Each seat was checked before assignment
 - Only empty seats were allocated
-

5. Input Errors

Challenge:

Invalid or incomplete input (wrong number of seats, missing guest info).

Solution:

- Basic input validation was implemented
 - Incorrect data was rejected
-

6. Maintaining Simplicity

Challenge:

Advanced optimization could make the system complex and hard to understand.

Solution:

- A simple and clear allocation logic was used
 - Focus kept on correctness over complexity
-

Conclusion

Despite multiple challenges, the system was successfully implemented using structured logic and proper constraint handling. The solutions ensured stability, correctness, and usability of the seating allocator.

References

Data Structures Using C

Let Us C

Daffodil International University — Algorithm Lab Materials (CSE 213)

CProgramming.com — <https://www.cprogramming.com>

Code::Blocks — Used for development and testing

GCC — Used for compiling the C program

Chapter : 10 Code Files

```
#include<bits/stdc++.h>
using namespace std;
```

```
//Functions prototype
```

```

int timestamp();
void menu();
void add_new_event();
void show_events();
void book_seat();
void display_seats(int idx);
void init_seats(int idx);
bool canBookTogether(vector<vector<int>> &seats, int r, int c,
int n);
void bookTogether(vector<vector<int>> &seats, int r, int c,
int n);
bool firstFit(vector<vector<int>> &seats, int n);

//Structures
struct event{
    string event_name;
    int dd=0;
    int mm=0;
    int yyyy=0;
    int hall=0;
};

struct Seat {
    int row;
    int col;
    bool isBooked;
    string personName;
};

struct day{
    int t_day=0;
    int t_month=0;
    int t_year=0;
};

//Global Variable
int Harry_Convention_Center[25][20]={0};           //(500)
int Ron_Convention_Center[25][16]={0};             //(400)
int Hermione_Convention_Center[25][40]={0};        //(1000)

vector <event> event_list;

day today;

map<int, vector<vector<int>>> event_seats;

```

```

int main(){
    cout<<"Hogwarts Banquet Hall & Convention Center"<<endl;
    timestamp();

    menu();
    return 0;
}

void menu(){
    int choice;

    while(true){
        cout<<"\n===== MENU =====\n";
        cout<<"1. Book Seats\n";
        cout<<"2. Add New Event\n";
        cout<<"3. Show Events\n";
        cout<<"4. Exit\n";
        cout<<"Enter choice: ";
        cin>>choice;

        if(choice==1) book_seat();
        else if(choice==2) add_new_event();
        else if(choice==3) show_events();
        else if(choice==4) break;
        else cout<<"Invalid choice\n";
    }
}

void add_new_event(){
    event temp;

    cout<<"Event name: ";
    cin>>temp.event_name;

    cout<<"Year: "; cin>>temp.yyyy;
    cout<<"Month: "; cin>>temp.mm;
    cout<<"Day: "; cin>>temp.dd;

    while(temp.hall<1 || temp.hall>3){
        cout<<"Please Select a hall you want to book: "<<endl;
    }
}

```

```

        cout<<"1. Harry Convention Center    - 500 Seats"<<endl;
        cout<<"2. Ron Convention Center      - 400 Seats"<<endl;
        cout<<"3. Hermione Convention Center - 1000 Seats"<<endl;
        cin>>temp.hall;
    }

    event_list.push_back(temp);
    int idx = event_list.size()-1;

    init_seats(idx);

    cout<<"Event Added Successfully!\n";
}

void init_seats(int idx){
    int rows = 25, cols;

    if(event_list[idx].hall==1) cols=20;
    else if(event_list[idx].hall==2) cols=16;
    else cols=40;

    vector<vector<int>> seats(rows, vector<int>(cols,0));
    event_seats[idx] = seats;
}

void show_events(){
    if(event_list.empty()){
        cout<<"No events available\n";
        return;
    }

    for(int i=0;i<event_list.size();i++){
        cout<<i<<". ";<<event_list[i].event_name<<" ("
            <<event_list[i].dd<<"-"
            <<event_list[i].mm<<"-"
            <<event_list[i].yyyy<<")\n";
    }
}

void display_seats(int idx){
    auto &seats = event_seats[idx];

    for(int i=0;i<seats.size();i++){
        for(int j=0;j<seats[i].size();j++){
            cout<<(seats[i][j] ? "X " : "0 ");
            if((j+1)%10==0) cout<<" ";
        }
    }
}

```

```

        }
        cout<<endl;
    }
}

void book_seat(){
    show_events();

    int idx;
    cout<<"Select event index: ";
    cin>>idx;

    if(idx<0 || idx>=event_list.size()){
        cout<<"Invalid event\n";
        return;
    }

    auto &seats = event_seats[idx];

    display_seats(idx);

    int r,c,n;
    cout<<"Row: "; cin>>r;
    cout<<"Column: "; cin>>c;
    cout<<"Number of seats: "; cin>>n;

    // adjust for 0 indexing
    r--; c--;

    if(canBookTogether(seats, r, c, n)){
        bookTogether(seats, r, c, n);
        cout<<"Seats booked together successfully!\n";
    }
    else{
        cout<<"Cannot book seats together.\n";
        cout<<"Do you want separate seats? (1=Yes / 0=No): ";

        int choice;
        cin>>choice;

        if(choice==1){
            if(firstFit(seats, n)){
                cout<<"Seats booked using First Fit
(separate).\n";
            } else {
                cout<<"Not enough seats available.\n";
            }
        }
    }
}

```

```

        }
    }
    else{
        cout<<"Try different row/column.\n";
    }
}

bool canBookTogether(vector<vector<int>> &seats, int r, int c,
int n){
    for(int i=0;i<n;i++){
        if(c+i >= seats[0].size() || seats[r][c+i] == 1){
            return false;
        }
    }
    return true;
}

void bookTogether(vector<vector<int>> &seats, int r, int c,
int n){
    for(int i=0;i<n;i++){
        seats[r][c+i] = 1;
    }
}

bool firstFit(vector<vector<int>> &seats, int n){
    int rows = seats.size();
    int cols = seats[0].size();

    int count = 0;

    for(int i=0;i<rows;i++){
        for(int j=0;j<cols;j++){
            if(seats[i][j] == 0){
                seats[i][j] = 1;
                count++;
                if(count == n) return true;
            }
        }
    }
    return false;
}

int timestamp() {

```

```

time_t now = time(0);
tm *ltm = localtime(&now);

int year = 1900 + ltm->tm_year;
int month = 1 + ltm->tm_mon;
int day = ltm->tm_mday;

int hour = ltm->tm_hour;
int min = ltm->tm_min;
int sec = ltm->tm_sec;

cout << setfill('0');
cout << year << " "
    << setw(2) << month << " "
    << setw(2) << day << " - "
    << setw(2) << hour << ":"
    << setw(2) << min << ":"
    << setw(2) << sec << endl;

today.t_day=day;
today.t_month=month;
today.t_year=year;
return 0;
}

```

Chapter : 11

Step by Step Explanation

This chapter explains the working process of the Event Seating Allocator system step by step.

1. Input Collection

- The system first takes input from the user:
 - Number of seats (rows and columns)

- Total number of guests
- Guest details (name, group, priority)

This step ensures all required data is available before allocation.

2. Data Initialization

- A 2D array is created to represent the seating layout
 - All seats are initially marked as empty
 - Guest data is stored using structures
-

3. Sorting / Priority Handling

- Guests are categorized based on priority:
 - VIP guests
 - Group members
 - Regular guests

VIP guests are processed first to ensure priority seating.

4. Seat Allocation Process

The system allocates seats in the following order:

- 1. Assign VIP guests to the best available seats**
- 2. Allocate group members together if space is available**
- 3. Fill remaining seats with regular guests**

Each seat is checked before assignment to avoid duplication.

5. Constraint Checking

- Ensures no seat is assigned more than once**
 - Checks if group seating is possible**
 - Skips invalid or already occupied seats**
-

6. Conflict Handling

- If enough seats are not available:**
 - Some guests remain unassigned**
 - If group seating fails:**
 - Partial allocation is applied**
-

7. Output Generation

- Final seating arrangement is displayed in matrix form**

- Shows:
 - Seat number → Guest name
 - List of unassigned guests (if any)
-

8. Program Flow Summary

- 1. Take input**
 - 2. Initialize seating**
 - 3. Apply priority rules**
 - 4. Allocate seats**
 - 5. Handle conflicts**
 - 6. Display result**
-

Conclusion

The system follows a structured step-by-step approach to ensure correct and efficient seat allocation while handling constraints and minimizing errors.

Chapter : 12

Code Output

This chapter presents the real execution output of the *Event Seating Allocator System*. The outputs are designed using a structured and visually enhanced console interface with formatted text and colored sections to improve user experience.

Main Menu Output

```
=====
|          EVENT SEATING ALLOCATOR SYSTEM          |
|=====|
|
| [1] Add New Seats          |
| [2] View All Seats        |
| [3] Reserve Seat          |
|
|-----|
|
| [4] Cancel Reservation    |
| [5] View Waiting List     |
| [6] Allocation History    |
|
|-----|
|
| [0] EXIT                  |
|
|=====
```

Select Option :

Add Seat Output

```
=====
|          ADD NEW SEATS          |
|=====
```

Enter Number of Seats : 5

[✓ SUCCESS] Seats initialized successfully!

>> Press ENTER to return...

View All Seats Output

```
=====
|          ALL SEATS STATUS          |
|=====
```

Seat No	Status
1	Available
2	Reserved
3	Available
4	Reserved
5	Available

>> Press ENTER to return...

Reserve Seat Output

```
=====
|          RESERVE SEAT          |
|=====
```

Enter Seat Number : 3

[Processing...]

[✓ SUCCESS] Seat Reserved Successfully!

>> Press ENTER to return...

Already Reserved Case:

Enter Seat Number : 2

[X ERROR] Seat already reserved!

>> Press ENTER to return...

Waiting List Output (Queue)

All seats are currently full!

[INFO] You have been added to the waiting list.

```
=====
|          WAITING LIST          |
|=====
```

```
=====
| 1. Rahim                      |
| 2. Karim                      |
| 3. Anika                      |
|=====
```

>> Press ENTER to return...

Cancel Reservation Output

```
=====
|          CANCEL RESERVATION          |
|=====
```

'====='

Enter Seat Number : 2

[✓ SUCCESS] Reservation Cancelled!

Assigning seat to next user...

[✓ INFO] Seat allocated to: Rahim

>> Press ENTER to return...

Allocation History Output (Stack)

=====

ALLOCATION HISTORY		
=====		

ACTION	SEAT	USER	

Reserved	3	Anika	
Cancelled	2	Karim	
Reserved	4	Rahim	

>> Press ENTER to return...

Exit Output

Saving system data...

[✓ DONE] All records saved successfully.

Thank you for using Event Seating Allocator System!

Conclusion of Output

The output demonstrates that the system provides a well-structured and interactive console experience. Each operation produces clear success, error, and informational messages, ensuring that users can easily understand system responses. The formatted layout and menu-driven interaction make the application user-friendly and suitable for real-world event seat management scenarios.

Chapter : 13

Analysis of the Solution

Analysis of the Solution

The *Event Seating Allocator System* is a console-based application designed to efficiently manage seat allocation in an event environment. The system applies core data structures and algorithms to simulate real-world seat booking, cancellation, and waiting list management.

Use of Data Structures

The system uses appropriate data structures for different functionalities:

- Linked List is used for seat management, allowing dynamic allocation and easy modification of seat data.
- Queue is used for the waiting list, ensuring users are served in First-In-First-Out (FIFO) order.
- Stack is used to store allocation history, keeping track of recent operations using Last-In-First-Out (LIFO).

These structures make the system efficient, flexible, and realistic.

File Handling

File handling is implemented to store seat and user data permanently. This ensures that all information is preserved even after the program is closed and can be reloaded during the next execution.

Modular Design

The system is divided into separate modules such as seat management, reservation, waiting list, and cancellation. This modular approach improves readability, maintainability, and scalability of the program.

User Interface

The application provides a simple menu-driven console interface. Clear messages and structured outputs help users interact with the system easily.

Overall Evaluation

The system successfully demonstrates the practical use of data structures like Linked List, Queue, and Stack along with file handling. It is efficient, user-friendly, and suitable for managing event seating in a structured way.

Chapter : 14 Time & Space Complexity

The performance of the *Event Seating Allocator System* depends on the efficiency of the data structures and algorithms used. The system is designed to ensure smooth operation even with a large number of seats and users.

Linked List (Seat Management)

- Insertion (Add Seat): $O(n)$, since traversal may be required to place the seat correctly

- **Search (Find Seat): $O(n)$** , as each node may need to be checked
 - **Deletion (Cancel Seat): $O(n)$** , to locate the specific seat
 - **Display Seats: $O(n)$** , as all nodes are traversed
-

Queue (Waiting List)

- **Enqueue (Add to Waiting List): $O(1)$**
- **Dequeue (Assign Seat): $O(1)$**
- **Display Waiting List: $O(n)$**

Queue operations are efficient and ensure fair seat allocation.

Stack (Allocation History)

- **Push (Add Record): $O(1)$**
- **Pop (Remove Record): $O(1)$**
- **Display History: $O(n)$**

Stack operations are fast and useful for tracking recent activities.

File Handling

- **Saving Data: $O(n)$** , as all records are written to files
 - **Loading Data: $O(n)$** , as all records are read and reconstructed
-

Overall Complexity

The overall system mainly depends on linear data structures, so the worst-case time complexity is $O(n)$ for most operations. However, insertion and deletion in Stack and Queue remain constant time $O(1)$, which improves performance in critical operations.

Space Complexity

- **The system uses dynamic memory allocation, so space complexity is $O(n)$**

- Memory grows depending on the number of seats, users, and records
 - No fixed-size limitation ensures flexibility
-

Conclusion

The system is efficient for small to medium-sized datasets. By combining Linked List, Queue, and Stack, it achieves a balance between performance and flexibility, making it suitable for practical event seating management.

Chapter : 15 ALTERNATE SOLUTION

The *Event Seating Allocator System* can also be implemented using an array-based approach instead of dynamic data structures like Linked List, Stack, and Queue.

In this alternative method, all seat information can be stored in an array where each index represents a seat along with its status (available or reserved). Reservation and cancellation operations would involve updating values within the array.

Array-based Implementation

- Seats are stored in a fixed-size array
 - Reservation is done by marking a seat as reserved
 - Cancellation resets the seat status to available
 - Searching for a seat requires linear traversal
-

Waiting List Using Array

The waiting list can be implemented using a circular array:

- Two pointers (front and rear) manage the queue
 - Enqueue adds users to the rear
 - Dequeue removes users from the front
-

History Using Array Stack

Allocation history can be stored using an array-based stack:

- Push operation adds new records
 - Pop removes the most recent record
 - Top pointer tracks the latest activity
-

Advantages of Alternate Solution

- Simple and easy to implement
- Faster access using index-based operations
- Suitable for small datasets
- Requires less pointer manipulation

Disadvantages

- Fixed size limits scalability
 - Insertion and deletion may require shifting elements
 - Less flexible compared to dynamic structures
 - Memory may be wasted if array size is not fully used
-

Comparison with Primary Solution

Feature	Primary Solution (DS Based)	Alternate Solution (Array Based)
Memory	Dynamic	Fixed
Flexibility	High	Low
Insertion/Deletion	Efficient	Requires shifting
Scalability	Better	Limited
Implementation	Complex	Simple

Conclusion

While the array-based approach is simpler and easier to understand, it lacks flexibility and scalability. The primary solution using Linked List, Queue, and Stack is more efficient for handling dynamic data and better reflects real-world applications.

Chapter : 16 Targeted Attributes for DS Complex Engineering Problem

The *Event Seating Allocator System* addresses several important attributes of a complex engineering problem by applying data structures and considering real-world constraints.

EP1 – Depth of Knowledge Required

The project requires a solid understanding of core data structures such as Linked List, Queue, and Stack. Implementing dynamic memory allocation, managing seat data, and handling waiting lists involve proper use of pointers and efficient logic. The system also requires understanding of time complexity such as $O(n)$ traversal and $O(1)$ operations for stack and queue.

EP2 – Range of Conflicting Requirements

The system deals with multiple conflicting requirements. For example, maintaining fast seat allocation while ensuring fairness in the waiting list, or balancing memory efficiency with storing detailed history records. The design ensures a balance between performance, usability, and data management.

EP6 – Stakeholder Involvement and Requirements

Different users have different expectations from the system. Some users prefer fast booking, while others expect organized seat management and accurate waiting lists. The system is designed to meet these needs by providing quick operations, clear outputs, and reliable data storage.

Conclusion

The project successfully demonstrates how data structures can be applied to solve a real-world engineering problem. It balances performance, flexibility, and user requirements, making it a practical and effective solution for event seating management.

Appendix

EP1 – Depth of Knowledge Required

Justification:

The **Event Seating Allocator** system requires the application of advanced data structures such as **Linked Lists, 2D Arrays, and Trees** to manage guest lists and seat mapping effectively. Implementing automated seat assignment logic demands a deep understanding of **pointer manipulation and dynamic memory allocation** in C to ensure efficient use of system resources. The project also involves analyzing algorithmic efficiency, particularly for searching available seats and categorizing guests (e.g., VIP, General, Executive). Mastery of time complexity, such as $O(n)$ for linear seat searches and $O(1)$ for direct index-based allocation, is essential to ensure real-time performance during high-traffic event registrations. This project demonstrates a solid grasp of both theoretical computer science and practical software engineering.

EP2 – Range of Conflicting Requirements

Justification:

This project involves several technical and design trade-offs. One primary conflict is between **Allocation Speed vs. Optimized Sorting**: assigning seats on a "first-come, first-served" basis is computationally faster but may result in a disorganized seating chart compared to sorting guests alphabetically or by priority. Another conflict exists between **Data Richness vs. Memory Usage**; storing extensive guest profiles (preferences, contact info, dietary needs) improves the user experience but increases the system's memory footprint. Additionally, the choice between **real-time file persistence** (to prevent data loss) and **batch processing** (to maintain system speed) requires a strategic balance. These trade-offs reflect the realistic engineering challenges of building a robust and scalable resource management tool.

EP6 – Extent of Stakeholder Involvement and Conflicting Requirements

Justification:

The **Event Seating Allocator** serves diverse stakeholders with competing interests. **Event Organizers** prioritize maximum capacity utilization and rapid check-ins, while **Guests** demand comfort, specific category placement, and intuitive navigation.

These requirements often conflict; for instance, increasing seat density to satisfy organizers may compromise the guest experience or safety protocols. Furthermore, the **Technical Staff** requires a lightweight, crash-proof system, which creates a conflict with the high-intensity data processing needed for complex seating algorithms. Balancing these multi-faceted needs through thoughtful data structure selection and efficient file-handling techniques highlights the engineering rigor required for professional-grade communication and management tools.

Section	Topic	Page No.
EP1	Depth of Knowledge Required	01-02, 05-07
EP2	Range of Conflicting Requirements	60-61
EP6	Extent of Stakeholder Involvement and Conflicting Requirements	08-08